

13.3 Avoiding the StackOverflowError

To better understand the `StackOverflowError`, consider this very simple program that demonstrates the problem:

```
val p = IO.forever(PrintLine("Still going..."))
```

If we evaluate `p.run`, it will crash with a `StackOverflowError` after printing a few thousand lines. If you look at the stack trace, you'll see that `run` is calling itself over and over. The problem is in the definition of `flatMap`:

```
def flatMap[B](f: A => IO[B]): IO[B] =
  new IO[B] { def run = f(self.run).run }
```

This method creates a new `IO` object whose `run` definition calls `run` again *before* calling `f`. This will keep building up nested `run` calls on the stack and eventually overflow it. What can be done about this?

13.3.1 Reifying control flow as data constructors

The answer is surprisingly simple. Instead of letting program control just flow through with function calls, we explicitly bake into our data type the control flow that we want to support. For example, instead of making `flatMap` a method that constructs a new `IO` in terms of `run`, we can just make it a data constructor of the `IO` data type. Then the interpreter can be a tail-recursive loop. Whenever it encounters a constructor like `FlatMap(x, k)`, it will simply interpret `x` and then call `k` on the result. Here's a new `IO` type that implements that idea.

Listing 13.4 Creating a new `IO` type

```
sealed trait IO[A] {
  def flatMap[B](f: A => IO[B]): IO[B] =
    FlatMap(this, f)
  def map[B](f: A => B): IO[B] =
    flatMap(f andThen (Return(_)))
}
case class Return[A](a: A) extends IO[A]
case class Suspend[A](resume: () => A) extends IO[A]
case class FlatMap[A,B](sub: IO[A], k: A => IO[B]) extends IO[B]
```

A pure computation that immediately returns an `A` without any further steps. When `run` sees this constructor, it knows the computation has finished.

A suspension of the computation where `resume` is a function that takes no arguments, but has some effect and yields a result.

A composition of two steps. Reifies `flatMap` as a data constructor rather than a function. When `run` sees this, it should first process the subcomputation `sub` and then continue with `k` once `sub` produces a result.

This new `IO` type has three data constructors, representing the three different kinds of control flow that we want the interpreter of this data type to support. `Return` represents an `IO` action that has finished, meaning that we want to return the value `a` without any further steps. `Suspend` means that we want to execute some effect to produce

a result. And the `FlatMap` data constructor lets us *extend* or *continue* an existing computation by using the result of the first computation to produce a second computation. The `flatMap` method's implementation can now simply call the `FlatMap` data constructor and return immediately. When the interpreter encounters `FlatMap(sub, k)`, it can interpret the subcomputation `sub` and then remember to call the continuation `k` on the result. Then `k` will continue executing the program.

We'll get to the interpreter shortly, but first let's rewrite our `println` example to use this new IO type:

```
def println(s: String): IO[Unit] =
  Suspend(() => Return(println(s)))

val p = IO.forever(println("Still going..."))
```

What this actually creates is an infinite nested structure, much like a `Stream`. The “head” of the stream is a `Function0`, and the rest of the computation is like the “tail”:

```
FlatMap(Suspend(() => println(s)),
  _ => FlatMap(Suspend(() => println(s)),
    _ => FlatMap(...)))
```

And here's the tail-recursive interpreter that traverses the structure and performs the effects:

Here `x` is a `Suspend(r)`, so we force the `r` thunk and call `f` on the result.

```
@annotation.tailrec def run[A](io: IO[A]): A = io match {
  case Return(a) => a
  case Suspend(r) => r()
  case FlatMap(x, f) => x match {
    case Return(a) => run(f(a))
    case Suspend(r) => run(f(r()))
    case FlatMap(y, g) => run(y flatMap (a => g(a) flatMap f))
  }
}
```

We could just say `run(f(run(x)))` here, but then the inner call to `run` wouldn't be in tail position. Instead, we match on `x` to see what it is.

In this case, `io` is an expression like `FlatMap(FlatMap(y, g), f)`. We reassociate this to the right in order to be able to call `run` in tail position, and the next iteration will match on `y`.

Note that instead of saying `run(f(run(x)))` in the `FlatMap(x, f)` case (thereby losing tail recursion), we instead pattern match on `x`, since it can only be one of three things. If it's a `Return`, we can just call `f` on the pure value inside. If it's a `Suspend`, then we can just execute its resumption, call `FlatMap` with `f` on its result, and recurse. But if `x` is itself a `FlatMap` constructor, then we know that `io` consists of two `FlatMap` constructors nested on the left like this: `FlatMap(FlatMap(y, g), f)`.

In order to continue running the program in that case, the next thing we naturally want to do is look at `y` to see if it is another `FlatMap` constructor, but the expression may be arbitrarily deep and we want to remain tail-recursive. We reassociate this to the right, effectively turning `(y flatMap g) flatMap f` into `y flatMap (a => g(a) flatMap f)`. We're just taking advantage of the monad associativity law! Then we call `run` on the rewritten expression, letting us remain tail-recursive. Thus, when we actually interpret